

Here is the complete inventory of the implemented modules, their file paths, purposes, and major feature details as they exist in the current repository.

1. IMPLEMENTED MODULES: FILE PATHS & PURPOSES

A. Backend Web Application (`backend/app/`)

- **main.py:** Bootstraps the FastAPI web server. Defines versioned API endpoints (`/api/v1/health`, `/runs`, `/resources`, `/recommendations`, `/approvals`), configures CORS middleware for frontend access, handles global exceptions (validation errors, invalid JWT tokens, unexpected system failures), initializes DB tables, and automatically triggers migrations/seeding.
- **database.py:** Initializes the SQLAlchemy database engine (SQLite) and manages local DB sessions. Exposes `get_db` dependency helper for FastAPI endpoints.
- **seed.py:** Populates the SQLite database. In `MOCK` mode, it inserts mock resources (VMs, disks, app service plans), CPU metric history, and audit records. In `LIVE` mode, it interfaces with the `LiveAzureClient` to dynamically discover active Azure VMs, disks, and App Service plans, fetch Monitor telemetry, and save them.
- **core/config.py:** Manages global configuration settings using `pydantic-settings`. Loads values from environment variables or a `.env` file at the project root, computes project root directories, defines validation rules for crucial settings (e.g. `JWT_SECRET_KEY`), and provides cached configuration access.
- **core/logging.py:** Sets up standard python logging configurations across the backend API, MCP server, and agent modules.

- **events/event_bus.py**: Implements a simple asynchronous Pub/Sub EventBus. Allows decoupled modules to publish event notifications and subscribe handlers (e.g., routing coordinator actions to database logging).
- **audit/__init__.py**: Empty module initialization for audit-specific logging packages.
- **models/base.py**: Defines the SQLAlchemy `Base` class and exposes a `generate_uuid` primary key generator.
- **models/run.py**: Defines SQLAlchemy ORM mappings for execution sweeps (`Run`) and agent action records (`AuditLog`).
- **models/resource.py**: Mapped DB schemas for tracked cloud resources (`Resource`) and their timeseries metrics history (`TelemetryHistory`).
- **models/recommendation.py**: ORM maps for cost savings proposals (`Recommendation`) and HITL verification tasks (`Approval`).
- **models/event.py**: Mapped schema for tracking specific system event notifications.
- **schemas/api.py**: Defines Pydantic validation request/response schemas (DTOs) for API interactions (e.g., triggering runs, granting approvals, resource payloads).
- **workflow/coordinator.py**: Implements the Workflow Coordinator state machine. Tracks step states, publishes events, handles failures and retries, inserts pending approvals, resumes execution upon token verification, and interfaces with the MCP tools.
- **workflow/models.py**: Exposes structural data classes representing

workflow state structures

(e.g. `WorkflowRun`, `WorkflowStep`, `ApprovalRequest`, `ActionPlan`, `StepStatus`, `WorkflowStatus`).

- **workflow/policy.py**: Defines the `RetryPolicy` model, managing retry limitations, initial delays, and exponential backoff factors.
- **workflow/queue.py**: Implements an in-memory queue (`ExecutionQueue`) for managing pending action plans.

B. Cloud Adapter & Integration (`cloud_adapter/`)

- **interface.py**: Outlines the `AzureClientAdapter` abstract base class defining compute lists, disks queries, telemetry pulls, Advisor recommendations, and remediation actions.
- **live_client.py**: Integrates with live Azure APIs via Python SDK (`DefaultAzureCredential`, `ComputeManagementClient`, `MonitorManagementClient`, etc.). Automatically falls back to the mock client if auth details are missing.
- **mock_client.py**: Simulates an active Azure environment. Contains in-memory structures representing VMs, disks, and plans, generating telemetry metrics with realistic variances.

C. Model Context Protocol Server (`mcp_server/`)

- **server.py**: Implements the Python FastMCP server. Exposes low-level cloud primitives to agent executors, signs/verifies JWT tokens, validates schemas, enforces dry-run safety, and fires async tool events.
- **exceptions.py**: Defines exceptions for FastMCP tooling (e.g., token errors,

execution errors).

- **metadata.py**: Defines validation models (`ToolMetadata`) tracking categories, permission constraints, and runtime characteristics of tools.

D. Kaggle/Google ADK Reasoning Agents (`agents/`)

- **shared/base_agent.py**: Defines `BaseAgent` abstract class requiring input/output schema definitions and execute overrides.
- **shared/context.py**: Structural layout for context objects passed through agent reasoning chains.
- **shared/enums.py**: Enum definitions representing message styles and trace statuses.
- **shared/message.py**: Encapsulates data transfer structures (`AgentMessage`) exchanged between reasoning agents.
- **shared/logger.py**: Formats logging outputs for thoughts and actions of AI agents.
- **shared/registry.py**: Implements `AgentRegistry` singleton to register, lookup, and list all active reasoning agents.
- **shared/adk_adapter.py**: Implements adapter wrappers that maps agent logic into the Google ADK execution styles.
- **shared/scenarios.py**: Stores pre-defined metrics payloads and configurations for all dry-run scenario tests.
- **shared/trace.py**: Defines trace classes (`AgentTrace`) for parent-child relationship mappings and execution performance indicators.

- **orchestrator/orchestrator.py:** Implements `GoogleADKOrchestrator`. Manages sessions, handles agent routing, context updates, and traces.
- **executive/agent.py:** Main coordinator agent (`ExecutiveOrchestratorAgent`). Deduce objectives, skip non-participating steps, manage retry attempts, and invoke tools.
- **telemetry/agent.py:** `TelemetryAgent` validates parameters and gathers performance metrics.
- **analysis/agent.py:** `AnalysisAgent` checks telemetry metrics to find idle VMs or unattached disks.
- **finops/agent.py:** `FinOpsAgent` calculates cost savings details.
- **policy/agent.py:** `PolicyAgent` checks resource tags and environments against compliance policies.
- **decision/agent.py:** `DecisionAgent` decides if recommended actions should be approved or require HITL sign-off.
- **execution/agent.py:** `ExecutionAgent` builds action plans and executes them.
- **verification/agent.py:** `VerificationAgent` checks post-execution resource states.
- **audit/agent.py:** `AuditAgent` creates audit logs.

E. Shared Utilities (`shared/`)

- **config.py:** Wrapper exposing centralized settings imports.

F. Frontend Dashboard Application (`frontend/src/`)

- **main.tsx**: Boots up the React application.
- **App.tsx**: Renders the base shell and navigates between views.
- **api/client.ts**: Typed client definition defining api endpoint mappings.
- **hooks/useWorkflow.ts**: Polling hook keeping local tables synchronized with backend API state.
- **components/DashboardLayout.tsx**: Standard layout containing links, title headers, and dark mode togglers.
- **views/Overview.tsx**: Panels presenting metrics, sweep metrics, and savings indicators.
- **views/WorkflowExecution.tsx**: Triggers autopilot runs and renders active state transitions.
- **views/Inventory.tsx**: Grid layouts listing active assets and visual cluster topology graphs.
- **views/Recommendations.tsx**: Active recommendations cards with manual approval triggers.
- **views/Explainability.tsx**: Reasoning chains inspect panel displaying stage findings and confidence scores.
- **views/AuditLogs.tsx**: Interactive list grid presenting all system audit events.

G. Infrastructure Layer (`infra/terraform/`)

- **main.tf:** Provisions Resource Group, VNet, Subnet, NSG, Standard Static Public IP, standard Ubuntu Linux VM, Storage Account, and Log Analytics Workspace.
- **variables.tf:** Setup inputs for configuration details.
- **outputs.tf:** Setup outputs for IDs, names, public IP, and password credentials.
- **terraform.tfvars:** Holds local input value configurations.

2. MAJOR FEATURES: CODE PATHS, HOW TO RUN, AND TODO/STUB INVENTORY

Feature 1: Multi-Agent Sweeps and Optimization Reasoning

- **Primary Source Files:**

- coordinator.py (Lines 679-695: `run_autopilot_reasoning` wrapper)
- agent.py (Lines 161-839: complete orchestrator reasoning steps)
- orchestrator.py (routing table registration & session logic)
- Specialized agents: `telemetry/agent.py`, `analysis/agent.py`, `finops/agent.py`, `policy/agent.py`, `decision/agent.py`.

- **How to Run:**

- **Trigger via API:** Make a `POST` request to `http://127.0.0.1:8000/api/v1/runs` with JSON body:

```
json{
  "scenario_name": "idle_vm",
  "dry_run": false
}
```

- **Trigger via UI:** Navigate to the **"Workflow Execution"** tab in the React Dashboard, select a scenario in the drop-down (e.g. `idle_vm`, `overprovisioned_vm`), and click **"Trigger Autopilot Run"**.
- **TODOs, Stubs, placeholders, or incomplete functionality:**
 - **None.** All agents are fully operational, evaluate inputs against real/mock data sources, and record results with full explainability.

Feature 2: Human-in-the-Loop Cryptographic Approval Gates (JWT Validation)

- **Primary Source Files:**
 - `server.py` (Lines 18-31: JWT helpers; Lines 397-486: `request_approval` tool; Lines 73-81: Tool permission checking)
 - `main.py` (Lines 310-360: `/approvals/{id}/approve` endpoint)
 - `coordinator.py` (Lines 339-443: Approval blocking/granting)
- **How to Run:**
 - When the workflow blocks (e.g. `blocked_on_approval` status), trigger a `POST` request to `http://127.0.0.1:8000/api/v1/approvals/{approval_id}/approve` with body `{"operator_id": "Admin"}`.
 - This generates an `HS256` signed token (valid for 10 minutes), commits the DB change, and passes it to the blocked Coordinator run, which decrypts it using the shared secret and resumes the execution step.
- **TODOs, Stubs, placeholders, or incomplete functionality:**
 - **None.**
 - *Backward Compatibility Feature:* To support legacy test models, there is a `get_compat_token` decorator wrapping raw tokens with a `token-` prefix. During JWT decoding, this prefix is automatically stripped. This is fully implemented and tested.

Feature 3: Live Azure SDK Integration and Fallback Simulations

- **Primary Source Files:**

- live_client.py
- mock_client.py

- **How to Run:**

- Controlled by the `CLOUD_MODE` setting in `d:\Project2\.env`.
- If `CLOUD_MODE=LIVE`, the `LiveAzureClient` attempts to log in using the `DefaultAzureCredential` and queries resource details under the subscription ID.
- If any SDK authentication fails, it logs a warning and automatically falls back to `MockAzureClient` logic to prevent system crashes.

- **TODOs, Stubs, placeholders, or incomplete functionality:**

- **Simulated Fallback:** In `live_client.py`, the fallback logic acts as a safe-mode proxy. If subscription credentials are not found (common during development), it redirects requests to the mock server instead of raising exceptions. This is the intended architecture design and is fully implemented.

Feature 4: Cloud Infrastructure Deployment (Terraform)

- **Primary Source Files:**

- main.tf
- outputs.tf

- **How to Run:**

- Navigate to the directory `d:\Project2\infra\terraform` and run:

```
bashterraform init
terraform apply -auto-approve
```

- **TODOs, Stubs, placeholders, or incomplete functionality:**
 - **None.** The outputs are fully active, and the virtual machine resource `cloudops-demo-vm` (IP: `40.80.88.74`) is active in Azure.